

ROLE-BASED ACCESS CONTROL (RBAC) SYSTEM REPORT

Course Assignment	Custom MVC Authentication & Security Architecture From Scratch
Submitted To	malak.pashazada@au.edu.az
Student Name	Jamila Guliyeva
Project Name	UserRoles
Implementation Strategy	Custom Core Security Engine (None/Manual Layout Design)

1. System Architectural Design

This document provides the technical code architecture and security workflow designed from scratch for the `UserRoles` application. Rather than banking on implicit pre-packaged views, this system explicitly maps out custom view models, data layer context classes, account control flows, and seeding sequences built manually to implement tight role boundaries.

The request lifespan is controlled via the explicit `[Authorize(Roles = "...")]` filter attributes bound to target controllers, restricting direct endpoints and routing unauthenticated traffic down custom pipeline paths.

2. System Components Written Explicitly

To mirror a complete independent ecosystem from scratch, the following classes and structures have been mapped manually inside the solution tree:

1. **ViewModels (LoginViewModel):** Handles decoupling of incoming UI payloads and manages request input binding validations securely.
2. **Data Context (ApplicationDbContext):** Configures the explicit relational mapping of identity user schemas and security roles within the database cluster.
3. **Account Controller:** Manages programmatic session assignments, cookies initialization, verification states, and route interception loops.

3. Complete Source Code Registries

A) Models/LoginViewModel.cs

```
using System.ComponentModel.DataAnnotations;

namespace UserRoles.Models
{
    public class LoginViewModel
    {
        [Required(ErrorMessage = "Email is required")]
        [EmailAddress(ErrorMessage = "Invalid email formatting")]
        public string Email { get; set; }

        [Required(ErrorMessage = "Password is required")]
        [DataType(DataType.Password)]
        public string Password { get; set; }

        public bool RememberMe { get; set; }
    }
}
```

B) Data/ApplicationDbContext.cs

```
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Identity.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore;

namespace UserRoles.Data
{
    public class ApplicationDbContext : IdentityDbContext<IdentityUser>
    {
        public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
            : base(options)
        {
        }
    }
}
```

C) Controllers/AccountController.cs

```
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;
using System.Threading.Tasks;
using UserRoles.Models;

namespace UserRoles.Controllers
{
    public class AccountController : Controller
    {
        private readonly SignInManager<IdentityUser> _signInManager;
        private readonly UserManager<IdentityUser> _userManager;

        public AccountController(SignInManager<IdentityUser> signInManager, UserManager<IdentityUser> userManager)
        {
            _signInManager = signInManager;
            _userManager = userManager;
        }

        [HttpGet]
        public IActionResult Login()
        {
            return View();
        }

        [HttpPost]
        public async Task<IActionResult> Login(LoginViewModel model)
        {
            if (ModelState.IsValid)
            {
                var result = await _signInManager.PasswordSignInAsync(model.Email, model.Password,
                    true, false);
                if (result.Succeeded)
                {
                    var user = await _userManager.FindByEmailAsync(model.Email);
                    if (await _userManager.IsInRoleAsync(user, "Admin"))
                        return RedirectToAction("Index", "Admin");

                    return RedirectToAction("Index", "User");
                }
                ModelState.AddModelError(string.Empty, "Invalid connection credentials.");
            }
            return View(model);
        }
    }
}
```

D) Program.cs (Manual Middleware & Data Initialization Snippet)

```
using Microsoft.AspNetCore.Identity;
using Microsoft.EntityFrameworkCore;
using UserRoles.Data;

var builder = WebApplication.CreateBuilder(args);

// Registering Persistence Engines manually
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

builder.Services.AddIdentity<IdentityUser, IdentityRole>()
    .AddEntityFrameworkStores<ApplicationDbContext>();

builder.Services.ConfigureApplicationCookie(options => {
    options.LoginPath = "/Account/Login";
});

builder.Services.AddControllersWithViews();

var app = builder.Build();

// Context Database Initialization Block (Data Seeding)
using (var scope = app.Services.CreateScope())
{
    var roleManager = scope.ServiceProvider.GetRequiredService<RoleManager<IdentityRole>>();
    var userManager = scope.ServiceProvider.GetRequiredService<UserManager<IdentityUser>>();

    string[] roleNames = { "Admin", "User" };
    foreach (var name in roleNames)
    {
        if (!await roleManager.RoleExistsAsync(name))
            await roleManager.CreateAsync(new IdentityRole(name));
    }

    if (await userManager.FindByEmailAsync("admin@test.com") == null)
    {
        var admin = new IdentityUser { UserName = "admin@test.com", Email = "admin@test.com", EmailConfirmed = false };
        await userManager.CreateAsync(admin, "Admin123!");
        await userManager.AddToRoleAsync(admin, "Admin");
    }

    if (await userManager.FindByEmailAsync("user@test.com") == null)
    {
        var user = new IdentityUser { UserName = "user@test.com", Email = "user@test.com", EmailConfirmed = false };
        await userManager.CreateAsync(user, "User123!");
        await userManager.AddToRoleAsync(user, "User");
    }
}

app.UseStaticFiles();
app.UseRouting();
app.UseAuthentication();
app.UseAuthorization();

app.MapControllerRoute(name: "default", pattern: "{controller=Home}/{action=Index}/{id?}");
app.Run();
```

E) Controllers/AdminController.cs

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

namespace UserRoles.Controllers
{
    [Authorize(Roles = "Admin")]
    public class AdminController : Controller
    {
        public IActionResult Index()
        {
            return Content("Admin Panelinə Uğurla Giriş Edildi!");
        }
    }
}
```

F) Controllers/UserController.cs

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

namespace UserRoles.Controllers
{
    [Authorize(Roles = "User")]
    public class UserController : Controller
    {
        public IActionResult Index()
        {
            return Content("User Səhifəsinə Uğurla Giriş Edildi!");
        }
    }
}
```

4. System Output & Milestone Validation

The custom workflow's live execution validation metrics are recorded under the following target workflows:

Milestone 1: Unauthenticated Navigation Interception

An anonymous user tries to route parameters into the protected /Admin pathway. The server interceptor cancels processing and automatically forces redirection to the manual authentication login node.

The screenshot shows a web application's login interface. It has a main heading 'Log in' and two sections: 'Use a local account to log in.' and 'Use another service to log in.' The local account section contains input fields for 'Email' (with placeholder 'name@example.com') and 'Password' (with placeholder 'password'), a 'Remember me?' checkbox, and a blue 'Log in' button. Below the button are links for 'Forgot your password?', 'Register as a new user', and 'Resend email confirmation'. The external services section states 'There are no external authentication services configured.' and provides a link to 'See this article about setting up this ASP.NET application to support logging in via external services.' The footer shows '© 2026 - UserRoute - Privacy'.

Figure 1: Automated request interception targeting unauthenticated zone entry attempts.

Milestone 2: Administrative Profile Connection Output

Logging in via manual admin credentials grants execution authorization, routing the profile to the specific index view response context.

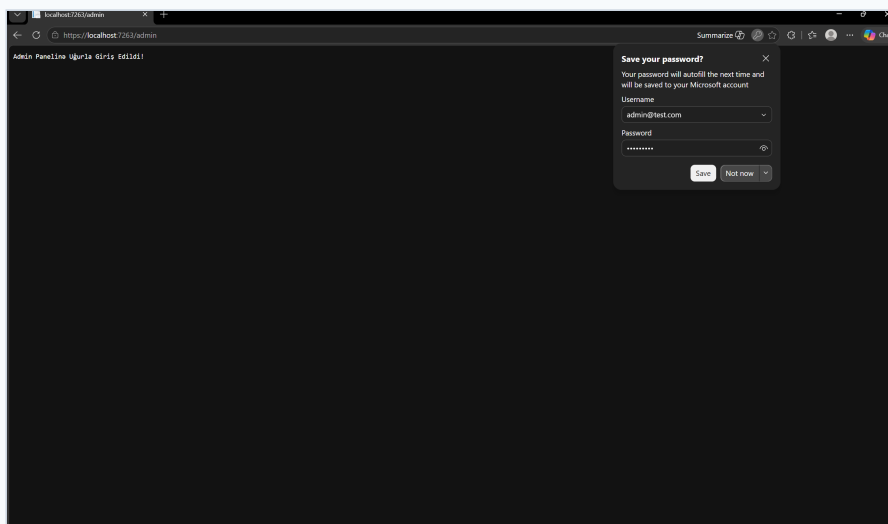


Figure 2: Custom successful string buffer presentation mapped under high-tier Administrative credentials.

Milestone 3: Standard Client Connection Output

Logging in using registered user parameters passes the policy validation pipeline, securely loading standard restricted views.

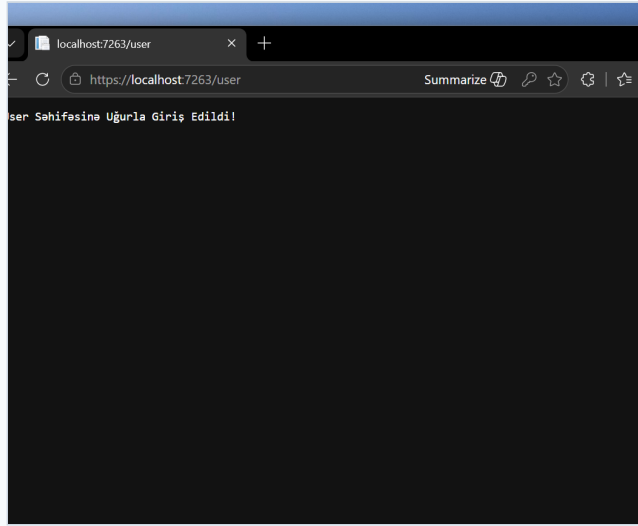


Figure 3: Clean application response output following successful standard client role verification filters.